

Récurtivité & structures de données avancées

Pr. Khadija SLIMANI

khadija.slimani@esiea.fr

PASS Java — Année universitaire 2026–2027

Plan du cours

Séance 5 — Récursivité

1. Principe de la récursivité
2. Cas de base et cas récursif
3. Exemple : factoriel
4. Algorithmes gloutons (greedy)
5. Diviser pour régner

Séance 6 — Structures de données avancées

6. La pile (Stack — LIFO)
7. La file (Queue — FIFO)
8. Quand utiliser l'une ou l'autre ?
9. Exemples LIFO : parenthèses, undo
10. Exemples FIFO : file d'attente, BFS

Séance 5

Récurtivité

Comprendre et écrire des fonctions qui s'appellent elles-mêmes.

Objectifs de la séquence

- Comprendre ce qu'est une fonction récursive et pourquoi elle fonctionne
- Identifier le cas de base et le cas récursif dans un pseudocode
- Visualiser la pile d'appels lors de l'exécution
- Écrire et tracer l'exécution de l'algorithme du factoriel
- Découvrir deux familles d'algorithmes : glouton et diviser pour régner

 **« Une fonction qui se résout en se rappelant elle-même, sur un problème plus petit. »**

Qu'est-ce que la récursivité ?

Une fonction est dite récursive lorsqu'elle s'appelle elle-même, directement ou indirectement, pour résoudre une version plus petite du même problème.

Analogie : des poupées russes

Chaque poupée contient une poupée plus petite, jusqu'à la plus petite qui ne s'ouvre plus (le cas de base).

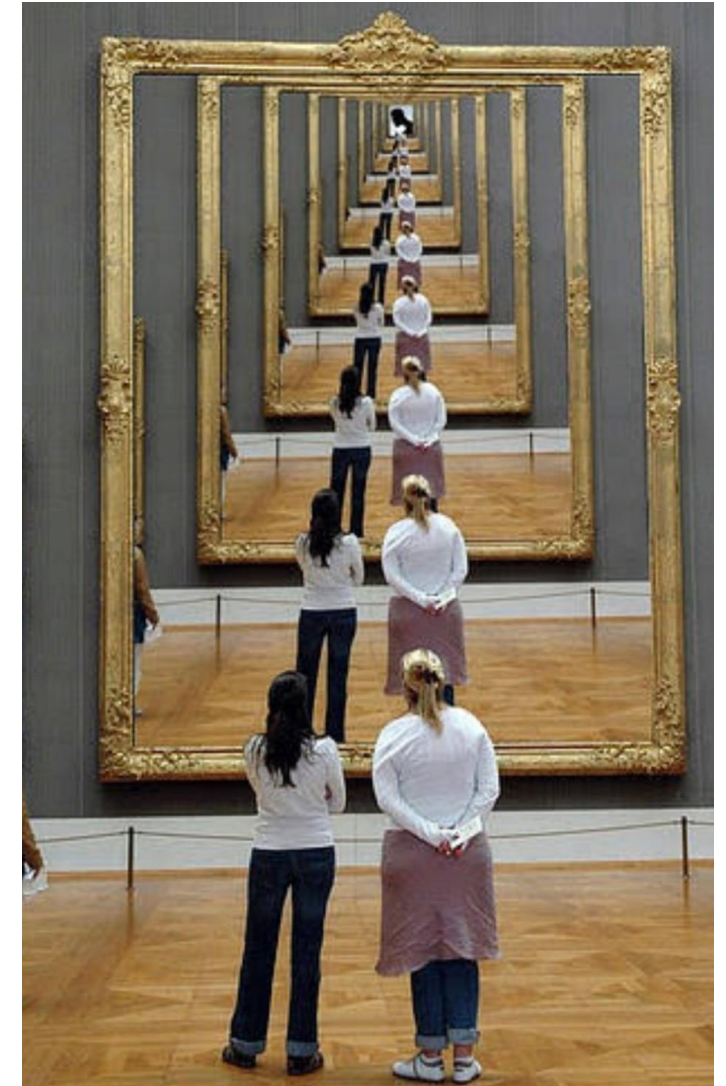
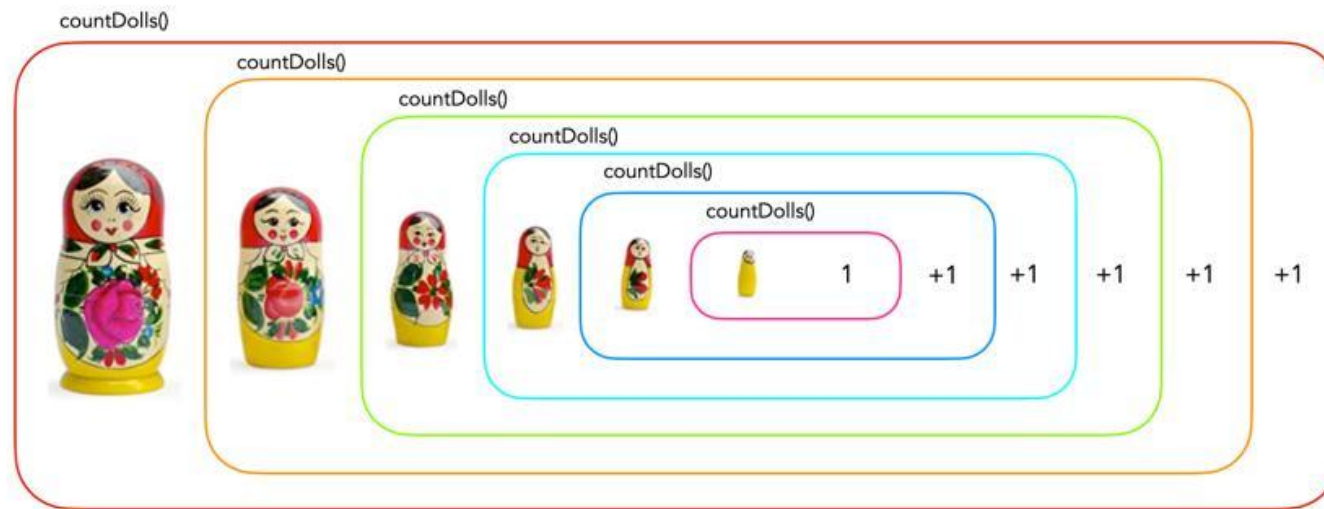
Analogie : deux miroirs face à face

Chaque reflet contient un reflet du reflet — sans limite, cela ne s'arrête jamais : c'est pour cela qu'un cas d'arrêt est indispensable.

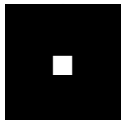


Qu'est-ce que la récursivité ?

- Poupées russes = récursion avec arrêt.
- Miroirs = récursion sans arrêt.



Anatomie d'une fonction récursive



Cas de base

La condition la plus simple, pour laquelle la fonction répond directement, sans nouvel appel. C'est lui qui arrête la récursion.



Cas récursif

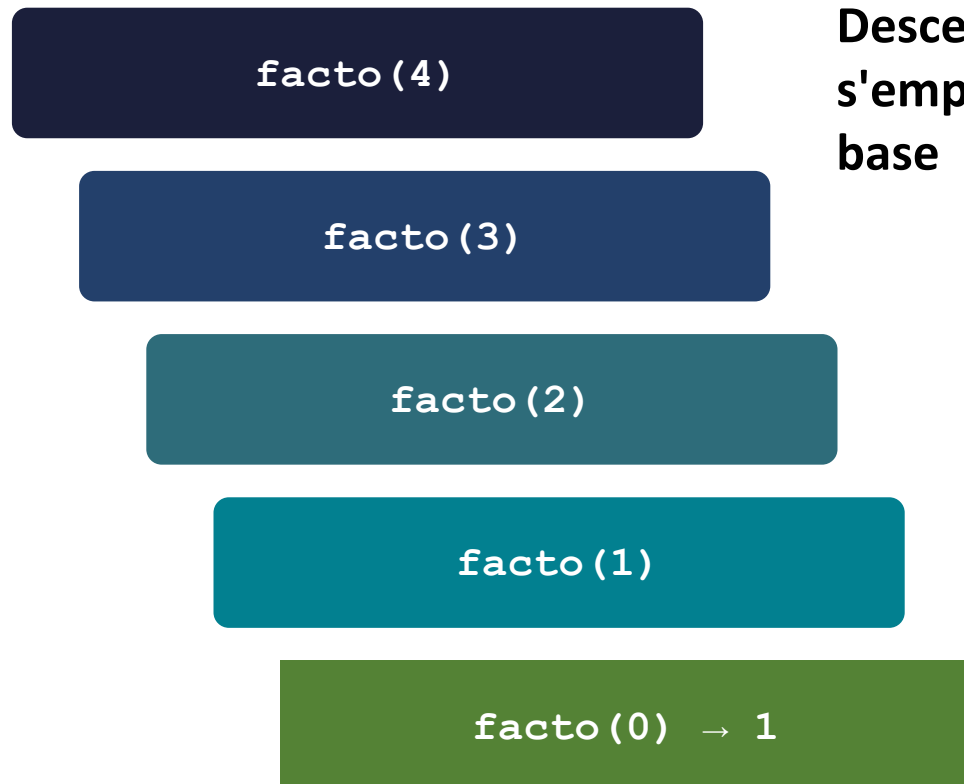
La fonction se rappelle elle-même sur une version plus petite ou plus simple du problème, en se rapprochant du cas de base.

```
fonction facto(n)
    si n == 0 alors                // cas de base
        retourner 1
    sinon                          // cas récursif
        retourner n * facto(n - 1)
    fin si
fin fonction
```

La pile d'appels (call stack)

- Chaque appel de fonction est empilé en mémoire, avec ses propres variables locales
- Un nouvel appel récursif ajoute un étage sur la pile, sans effacer les précédents
- Quand le cas de base est atteint, les appels se terminent un par un, du sommet vers la base
- Chaque étage « attend » le résultat de l'étage au-dessus avant de calculer le sien
- Trop d'appels imbriqués sans cas de base → dépassement de pile (stack overflow)

Visualiser la pile d'appels — facto(4)



Descente : les appels s'empilent jusqu'au cas de base

Remontée (dépilement)

$$\text{facto}(0) = 1$$

$$\text{facto}(1) = 1 \times 1 = 1$$

$$\text{facto}(2) = 2 \times 1 = 2$$

$$\text{facto}(3) = 3 \times 2 = 6$$

$$\text{facto}(4) = 4 \times 6 = 24$$

Le factoriel : définition

$$n! = n \times (n - 1) \times (n - 2) \times \dots \times 1$$

avec, par convention, $0! = 1$

$$4! = 4 \times 3 \times 2 \times 1 = 24$$

$$3! = 3 \times 2 \times 1 = 6$$

$$0! = 1 \text{ (cas de base)}$$

On observe que $n!$ peut se définir à partir de $(n - 1)!$: c'est cette relation qui rend le factoriel naturellement récursif.

Le factoriel : pseudocode

- La relation $n! = n \times (n - 1)!$ se traduit directement en pseudocode
- Le cas de base ($n == 0$) empêche la récursion infinie
- Chaque appel récursif se fait sur un problème strictement plus petit ($n - 1$)
- Le résultat final se construit lors de la remontée de la pile d'appels;

```
fonction facto(n)
    si n == 0 alors
        retourner 1
    sinon
        retourner n * facto(n - 1)
    fin si
fin fonction

// Appel : afficher facto(4)
// affiche 24
```

Trace d'exécution de facto(4)

Appel	Condition	Action
facto(4)	4 ≠ 0	retourne 4 * facto(3)
facto(3)	3 ≠ 0	retourne 3 * facto(2)
facto(2)	2 ≠ 0	retourne 2 * facto(1)
facto(1)	1 ≠ 0	retourne 1 * facto(0)
facto(0)	0 == 0	retourne 1 (cas de base)

Puis la remontée :

1 * 1 = 1

2 * 1 = 2

3 * 2 = 6

4 * 6 = 24

Résultat final : facto(4) = 24

Réversivité vs itération

Réversivité

- Code souvent plus court et plus lisible
- Naturelle pour les problèmes définis récursivement (arbres, factoriel)
- Coût mémoire : chaque appel occupe de la pile
- Risque de dépassement de pile si trop d'appels

Itération (boucle)

- Utilise une boucle (tant que / pour)
- Consomme peu de mémoire supplémentaire
- Parfois moins lisible pour des problèmes naturellement récursifs
- Toujours possible de transformer une récursion en boucle

Au-delà du factoriel : deux grandes familles d'algorithmes

La récursivité est aussi le mécanisme derrière de nombreux algorithmes classiques.

Algorithme glouton (greedy) : définition

- Principe : à chaque étape, on choisit l'option qui semble la meilleure sur l'instant, sans revenir en arrière
- « Glouton » car l'algorithme prend toujours le choix le plus avantageux immédiatement
- Simple et rapide à mettre en œuvre
- Ne garantit pas toujours la solution optimale globale (voir plus loin)
- Exemple typique : le rendu de monnaie avec le moins de pièces possible

Exemple glouton : le rendu de monnaie

- But : rendre une somme avec le moins de pièces possible
- À chaque étape, on prend la plus grande pièce possible qui ne dépasse pas la somme restante
- On répète jusqu'à ce que la somme restante soit nulle
- Version récursive : chaque appel traite une pièce, puis se rappelle sur le reste

```
fonction rendu_monnaie(somme, pieces)
    si somme == 0 alors
        retourner liste_vide
    p ← plus_grande_piece(pieces, somme)
    retourner [p] + rendu_monnaie(somme -
p, pieces)
fin fonction
```

Trace : rendre 67 centimes

Pièces disponibles : 50, 20, 10, 5, 2, 1 (centimes)

Somme restante	Pièce choisie	Nouvelle somme
67	50	17
17	10	7
7	5	2
2	2	0

Résultat

$$50 + 10 + 5 + 2 = 67$$

4 pièces utilisées

Les limites de l'approche gloutonne

Le glouton ne garantit pas toujours la solution optimale : un bon choix local peut mener à un mauvais résultat global.

Contre-exemple

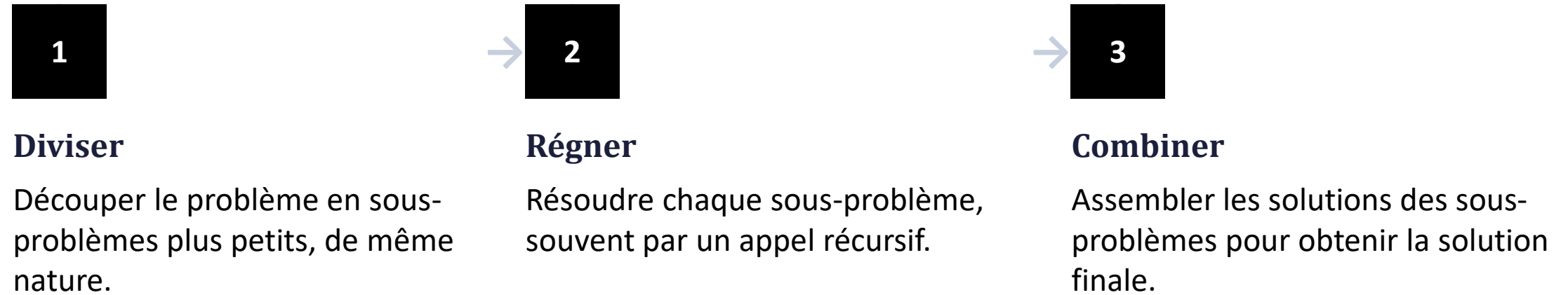
Pièces disponibles : 1, 3, 4 — objectif : rendre 6

Glouton choisit 4, puis 1, puis 1, puis 1 $\rightarrow$ 4 pièces (4+1+1)

Solution optimale : 3 + 3 $\rightarrow$ seulement 2 pièces;

À retenir : le glouton est rapide, mais il faut vérifier qu'il convient au problème posé.

Diviser pour régner (divide and conquer)



Exemples classiques : recherche dichotomique, tri fusion (merge sort), tri rapide (quicksort).

Comme le factoriel, ces algorithmes s'expriment naturellement de façon récursive : chaque sous-problème est résolu par le même algorithme.

Exemple : la recherche dichotomique

- **But** : trouver un élément dans une liste triée
- **Diviser** : comparer la cible à l'élément du milieu
- **Régner** : chercher récursivement dans la moitié gauche ou droite
- **Combiner** : ici, rien à combiner — la réponse remonte directement.

```
fonction recherche(liste, cible, debut, fin)
    si debut > fin alors
        retourner -1          // non trouvé
    milieu ← (debut + fin) / 2
    si liste[milieu] == cible alors
        retourner milieu
    sinon si liste[milieu] > cible alors
        retourner recherche(liste, cible, debut, milieu -
1)
    sinon
        retourner recherche(liste, cible, milieu + 1,
fin)
    fin si
fin fonction
```

Trace : chercher 23 dans une liste triée

3	7	12	18	23	29	34	41
0	1	2	3	4	5	6	7

Étape	Comparaison	Décision
Étape 1	<code>milieu = 3 → liste[3] = 18</code>	<code>18 < 23 → chercher à droite</code>
Étape 2	<code>milieu = 5 → liste[5] = 29</code>	<code>29 > 23 → chercher à gauche</code>
Étape 3	<code>milieu = 4 → liste[4] = 23</code>	trouvé ! index 4

Complexité : à chaque étape, on élimine la moitié des éléments restants → très efficace sur de grandes listes triées.

Ce qu'il faut retenir

Réursivité

Une fonction qui s'appelle elle-même, guidée par un cas de base et un cas récursif.

Pile d'appels

Chaque appel s'empile en mémoire ; la remontée construit le résultat final.

Glouton

Choix localement optimal à chaque étape — rapide, mais pas toujours optimal globalement.

Diviser pour régner

Diviser, régner (récursivement), combiner — base de nombreux algorithmes efficaces.

Séance 6

Structures de données avancées

La pile (LIFO) et la file (FIFO) : principes, opérations et usages.

Objectifs de la séquence

- Comprendre le fonctionnement d'une pile (LIFO) et d'une file (FIFO).
- Connaître les opérations de base de chaque structure.
- Savoir dans quels cas utiliser une pile plutôt qu'une file, et inversement.
- Étudier deux exemples LIFO : parenthèses équilibrées, fonction annuler (undo).
- Étudier deux exemples FIFO : file d'attente, parcours en largeur (BFS).

☰ « *Deux structures très simples, utilisées partout : pile d'annulation, files d'attente, navigateurs web... »*

Qu'est-ce qu'une structure de données ?

- Une structure de données organise et stocke des informations pour les manipuler efficacement.
- Elle définit surtout l'ordre dans lequel les éléments sont ajoutés et retirés.
- **Deux structures fondamentales à connaître aujourd'hui :**
 - la pile (Stack), qui suit la règle LIFO.
 - la file (Queue), qui suit la règle FIFO.

La pile (Stack) — LIFO

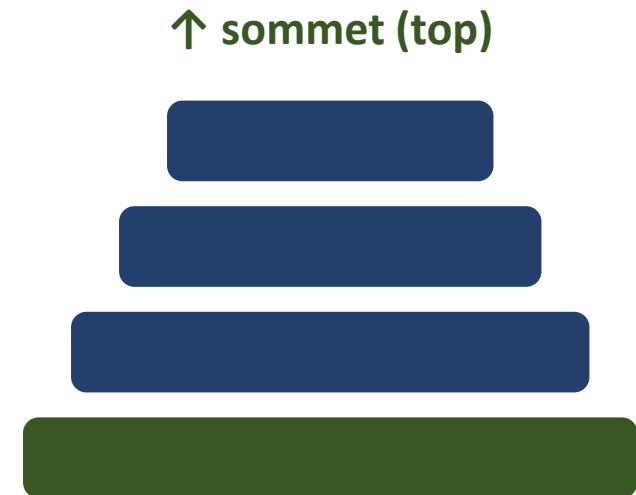
LIFO = Last In, First Out

« Dernier arrivé, premier sorti »

Le dernier élément ajouté est toujours le premier retiré.

Analogie : une pile d'assiettes

On pose les assiettes propres sur le dessus de la pile, et on reprend toujours l'assiette du dessus en premier.



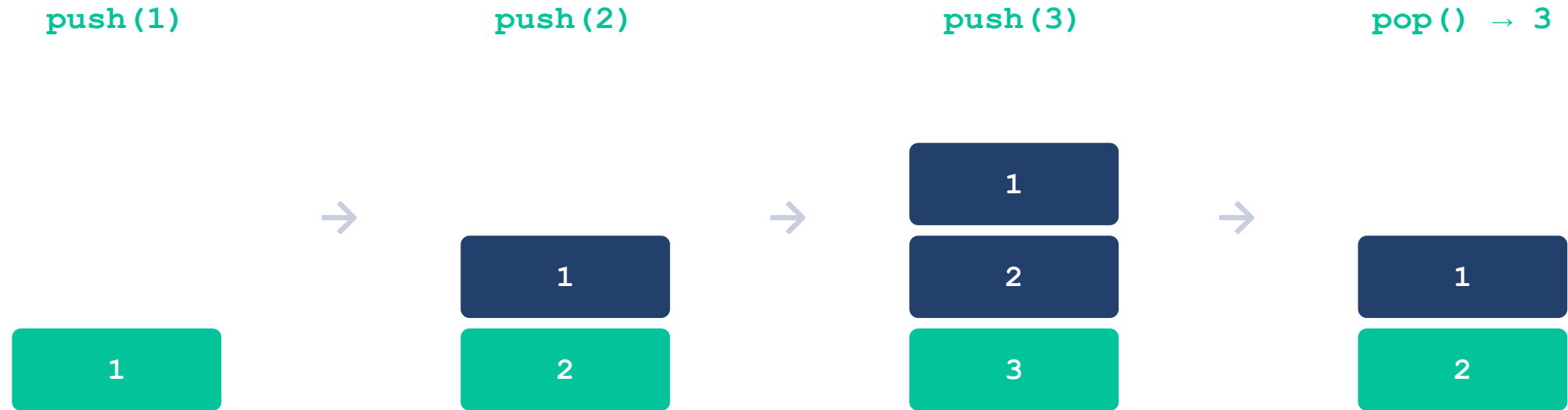
Les opérations sur la pile

- **push(x)** : ajoute l'élément x au sommet de la pile.
- **pop()** : retire et retourne l'élément du Sommet.
- **peek() / top()** : consulte l'élément du sommet sans le retirer.
- **est_vide()** : indique si la pile ne contient aucun élément.
- Toutes ces opérations se font en temps constant.

```
pile ← nouvelle_pile()
pile.push(1)
pile.push(2)
pile.push(3)           // pile = [1, 2, 3]

pile.pop()             // retire 3
pile.peek()            // lit 2, ne retire rien
                        // pile = [1, 2]
```

Schéma : empiler puis dépiler



Le sommet de la pile (en couleur) est toujours le prochain élément à sortir.

Pourquoi et quand utiliser une pile ?

- Chaque fois que l'ordre « dernier entré, premier sorti » a du sens
- Vérifier que des symboles sont bien équilibrés (parenthèses, balises)
- Annuler la dernière action effectuée (fonction « undo »)
- Gérer les appels de fonctions (la pile d'appels, vue en séquence 5 !)
- Explorer un graphe ou un labyrinthe en profondeur (parcours en profondeur, DFS)
- Revenir en arrière dans l'historique de navigation d'un navigateur web

Vérifier des parenthèses équilibrées

- But : vérifier qu'une expression comme « (a + [b * c]) » est bien parenthésée
- On empile chaque parenthèse ouvrante rencontrée
- À chaque parenthèse fermante, on dépile et on vérifie la correspondance
- À la fin, la pile doit être vide pour que l'expression soit valide

```
fonction parentheses_ok(expr)
  pile ← nouvelle_pile()
  pour chaque caractere c dans expr
    si c est une ouvrante alors
      pile.push(c)
    sinon si c est une fermante alors
      si pile.est_vide() alors
        retourner faux // fermante en trop
      pile.pop()
  retourner pile.est_vide() // rien ne
doit rester
fin fonction
```

Trace : vérifier « (a [b] c) »

Caractère	Action	État de la pile
(	<code>pile.push(()</code>	(
a	rien à faire	(
[	<code>pile.push([]</code>	([
b	rien à faire	([
]	<code>pile.pop()</code> → [correspond	(
c	rien à faire	(
)	<code>pile.pop()</code> → (correspond	(vide)

Pile vide à la fin → l'expression est bien équilibrée ✓

La fonction « annuler » (undo)

- Chaque action effectuée est empilée dans une pile d'historique
- Quand l'utilisateur clique sur « Annuler », on dépile la dernière action
- On l'annule, ce qui restaure l'état juste avant cette action
- C'est le principe LIFO : on annule toujours l'action la plus récente en premier

```
historique ← nouvelle_pile()

fonction effectuer(action)
    appliquer(action)
    historique.push(action)
fin fonction

fonction annuler()
    si non historique.est_vide() alors
        derniere ← historique.pop()
        inverser(derniere)           // restaure
l'état précédent
fin fonction
```

La file (Queue) — FIFO

FIFO = First In, First Out

« Premier arrivé, premier sorti »

Le premier élément ajouté est toujours le premier retiré.

Analogie : une file d'attente au guichet

La première personne arrivée dans la file est aussi la première servie ; les nouveaux arrivants se placent à la fin.



Les opérations sur la file

- **enqueue(x)** : ajoute l'élément x à la fin de la file
- **dequeue()** : retire et retourne l'élément du début de la file
- **peek() / front()** : consulte le premier élément sans le retirer
- **est_vide()** : indique si la file ne contient aucun élément
- Toutes ces opérations se font en temps constant

```
file ← nouvelle_file()
file.enqueue(1)
file.enqueue(2)
file.enqueue(3)      // file = [1, 2, 3]

file.dequeue()       // retire 1 (le premier)
file.peek()          // lit 2, ne retire rien
                     // file = [2, 3]
```

Schéma : enfiler puis défiler



La tête de la file (en couleur) est toujours le prochain élément à sortir.

Pourquoi et quand utiliser une file ?

- Chaque fois que l'ordre « premier entré, premier sorti » a du sens
- Gérer une file d'attente réelle : impression de documents, tickets de caisse
- Traiter des tâches dans leur ordre d'arrivée (files de messages, tâches en attente)
- Explorer un graphe ou un labyrinthe niveau par niveau (parcours en largeur, BFS)
- Gérer des requêtes réseau ou des appels dans l'ordre où ils arrivent

La file d'attente au guichet

- Chaque client qui arrive est ajouté à la fin de la file
- Le guichet appelle toujours le client en tête de file
- C'est le principe FIFO : premier arrivé, premier servi
- Aucun client n'est jamais « doublé » par un client arrivé après lui

```
file ← nouvelle_file()

fonction client_arrive(nom)
    file.enqueue(nom)
fin fonction

fonction guichet_libre()
    si non file.est_vide() alors
        client ← file.dequeue()
        servir(client)
    fin fonction
```

Le parcours en largeur (BFS)

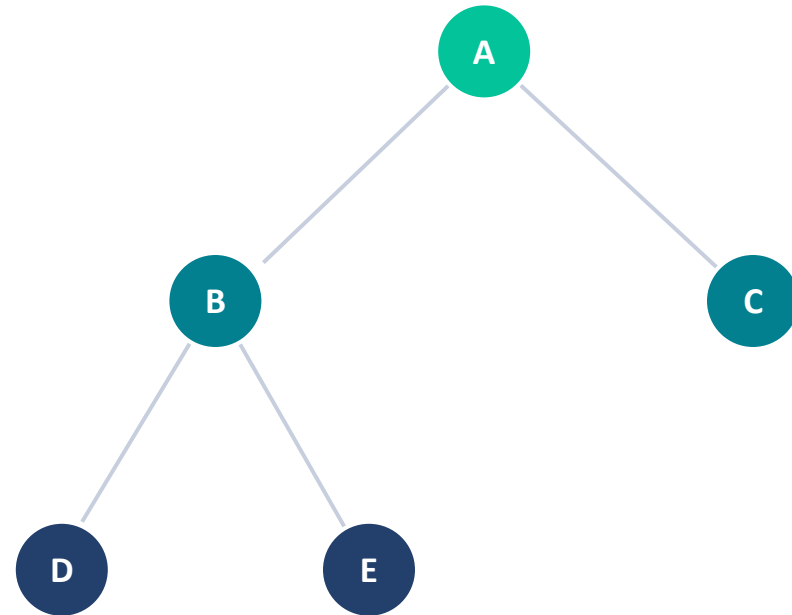
Le BFS (Breadth-First Search) explore un graphe niveau par niveau, en s'éloignant progressivement du point de départ.

Une file mémorise les nœuds à visiter, dans l'ordre où ils sont découverts.

Niveau 0

Niveau 1

Niveau 2



Ordre de visite BFS : A, B, C, D, E — la file garantit qu'on termine chaque niveau avant de passer au suivant.

BFS : pseudocode

- On part d'un nœud de départ, mis dans la file et marqué comme visité
- Tant que la file n'est pas vide, on défile un nœud et on visite ses voisins
- Chaque voisin non visité est marqué puis enfilé à son tour
- La file garantit un traitement niveau par niveau

```
fonction bfs(graphe, depart)
    file ← nouvelle_file()
    visites ← ensemble_vide
    file.enqueue(depart)
    visites.ajouter(depart)
    tant que non file.est_vide()
        noeud ← file.dequeue()
        afficher(noeud)
        pour chaque voisin dans graphe[noeud]
            si voisin non dans visites alors
                visites.ajouter(voisin)
                file.enqueue(voisin)
    fin fonction
```

Pile vs file : tableau récapitulatif

	Pile (Stack)	File (Queue)
Règle	LIFO — dernier entré, premier sorti	FIFO — premier entré, premier sorti
Ajout	push()	enqueue()
Retrait	pop()	dequeue()
Analogie	pile d'assiettes	file d'attente
Exemples typiques	undo, parenthèses, DFS, appels de fonctions	file d'attente, tâches en attente, BFS

Exercices d'application